Copy-on-Write String - Shared Reference Counting

Theory, Edge Cases & Code Examples

- ✓ Copy-on-Write Concept
- ✓ Reference Counting
- ✓ Multithreaded Detach Race
- ✓ Iterator Invalidation
- ✓ Cheap Copies

→ Swipe to learn →

Copy-on-Write String - Shared Reference Counting

Theory

📌 1. Copy-on-Write Concept

Idea: Share data until modification, then copy.

Benefits:

- Cheap copies (no allocation)
- Lazy duplication (only when needed)

Use cases:

- Strings (old std::string implementations)
- Large buffers
- Fork() in Unix

Copy-on-Write String - Shared Reference Counting

Theory

📌 2. Reference Counting (1/3)

Track how many strings share the same buffer:

```
String s1 = "hello";  // refcount = 1
String s2 = s1;        // refcount = 2 (shared)
s2[0] = 'H';          // Copy buffer (refcount s1=1, s2=1)
```

```
#include <cstring>
#include <atomic>
#include <iostream>

class COWString {
private:
    struct Buffer {
        std::atomic<int> refcount{1};
        size_t length;
        char data[1];  // Flexible array member

        static Buffer* create(const char* str, size_t len) {
            Buffer* buf = static_cast<Buffer*>(
                ::operator new(sizeof(Buffer) + len)
            );

            buf->refcount.store(1);
            buf->length = len;
            std::memcpy(buf->data, str, len + 1);  // +1 for null

            return buf;
        }

        void addRef() {
            refcount.fetch_add(1, std::memory_order_relaxed);
        }

        void release() {
            if (refcount.fetch_sub(1, std::memory_order_release) == 1) {
                std::atomic_thread_fence(std::memory_order_acquire);
                ::operator delete(this);
            }
        }
// ...continues on next slide
```

Copy-on-Write String - Shared Reference Counting

Theory

📌 2. Reference Counting (2/3)

```
// ...continued from previous slide
    };

    Buffer* buffer_;

    void detach() {
        if (buffer_->refcount.load(std::memory_order_relaxed) > 1) {
            // Copy buffer
            Buffer* new_buf = Buffer::create(buffer_->data, buffer_->length);
            buffer_->release();
            buffer_ = new_buf;
        }
    }

public:
    COWString(const char* str = "")
        : buffer_(Buffer::create(str, std::strlen(str)))
    {}

    COWString(const COWString& other)
        : buffer_(other.buffer_)
    {
        buffer_->addRef();
    }

    COWString& operator=(const COWString& other) {
        if (this != &other) {
            buffer_->release();
            buffer_ = other.buffer_;
            buffer_->addRef();
        }
        return *this;
    }

    ~COWString() {
        buffer_->release();
    }

    const char* c_str() const {
        return buffer_->data;
    }
// ...continues on next slide
```

Copy-on-Write String - Shared Reference Counting

Theory

📌 2. Reference Counting (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    size_t length() const {
        return buffer_->length;
    }

    char& operator[](size_t index) {
        detach();  // Copy-on-write!
        return buffer_->data[index];
    }

    const char& operator[](size_t index) const {
        return buffer_->data[index];
    }
};
```

Copy-on-Write String - Shared Reference Counting

Edge Cases

📌 1. Multithreaded Detach Race

```
// Thread 1:
s1[0] = 'A';  // Detaches

// Thread 2:
s2[0] = 'B';  // Also detaches

// Race on refcount!
```

Solution: Atomic refcount.

Copy-on-Write String - Shared Reference Counting

Edge Cases

📌 2. Iterator Invalidation

```
COWString s = "hello";
const char* ptr = s.c_str();

COWString s2 = s;  // Shares buffer
s2[0] = 'H';       // Detaches s2

// ptr still points to old buffer (valid for s, not s2)
```

Copy-on-Write String - Shared Reference Counting

Code Examples

📌 1. Cheap Copies (1/2)

This example demonstrates **the core advantage of Copy-on-Write strings: zero-cost copying until modification is needed**. The code shows how multiple string copies can share the same underlying buffer (using reference counting), and only when one string is modified does a buffer duplication occur - the "copy-on-write" moment.

What this code does:

- Creates `s1` with "hello, world!" - allocates buffer with refcount=1
- Copies `s1` to `s2` using copy constructor - increments refcount to 2 (no new allocation!)
- Both strings share the same buffer, making the copy O(1) constant-time and memory-efficient
- Modifying `s2[0] = 'H'` triggers `detach()` - now refcount check shows sharing, so buffer is duplicated
- After detach: `s1` has refcount=1 with original buffer, `s2` has refcount=1 with modified buffer
- Both strings are now independent and can be modified without affecting each other

Key concepts demonstrated:

- **Reference counting** - atomic integer tracks how many strings share the buffer
- **Lazy copying** - allocation is deferred until absolutely necessary (write operation)
- **Shared immutability** - read-only operations (like `c_str()`) never trigger copying
- **Detach-on-write** - the `operator[]` non-const version checks refcount and copies if > 1
- **Memory efficiency** - passing strings by value to functions doesn't allocate if function only reads

Why this matters:

Copy-on-Write was used in many pre-C++11 std::string implementations because **most string operations are read-only**. In typical code, strings are copied frequently (passed to functions, returned from functions, stored in containers) but rarely modified. COW made these copies virtually free - just incrementing a refcount - saving both memory and time. This was especially valuable when strings were large documents or configuration data.

Performance implications:

- Copy constructor: O(1) instead of O(n) - just increment atomic refcount
- Assignment operator: O(1) for assignment, O(n) only when first modified
- Memory savings: 1000 copies of "config.txt" content share 1 buffer instead of 1000 separate buffers
- Detach cost: O(n) allocation and memcpy when write occurs, but amortized if many reads precede it
- Thread-safety overhead: atomic refcount operations add ~10-20 CPU cycles vs non-atomic

Why COW is no longer used in modern C++:

C++11 introduced **move semantics**, which provides even cheaper transfers (literally stealing the buffer pointer) without the thread-safety overhead of atomic refcounts. Small String Optimization (SSO) also makes small strings free from allocation entirely. COW's unpredictable detach behavior was deemed incompatible with modern C++ performance expectations.

Output:

Copy-on-Write String - Shared Reference Counting

Code Examples

📌 1. Cheap Copies (2/2)

```
#include <iostream>

int main() {
    COWString s1 = "hello, world!";

    std::cout << "s1: " << s1.c_str() << '\n';

    COWString s2 = s1;  // No allocation! Shares buffer

    std::cout << "s2: " << s2.c_str() << '\n';

    s2[0] = 'H';  // NOW copies (detach)

    std::cout << "After modify:\n";
    std::cout << "s1: " << s1.c_str() << '\n';  // hello
    std::cout << "s2: " << s2.c_str() << '\n';  // Hello

    return 0;
}
```

```
['s1: hello, world!\ns2: hello, world!\nAfter modify:\ns1: hello, world!\ns2: Hello, world!']
```

What You Learned

Key Takeaways

- ✓ Copy-on-Write Concept
- ✓ Reference Counting
- ✓ Multithreaded Detach Race
- ✓ Iterator Invalidation

→ Swipe to learn →